

ConfigReader - Detailed Explanation (with Code)

What is this class?

ConfigReader is a utility class placed in the utils package. Its purpose is to read configuration values from a config.properties file so that we don't hardcode values (like URLs, usernames, passwords) in our test code. It is static-based, so we can call methods directly without creating an object.

```
// Usage
ConfigReader.loadConfig("dev");
String baseUrl = ConfigReader.getProperty("base.url");
```

Properties field

The Properties class is a built-in Java class from java.util. It works like a HashMap but is designed for config files (.properties).

```
private static Properties properties;
```

```
// Example config.properties
base.url=https://example.com
browser=chrome
timeout=10
```

loadConfig(String env) method

This is a static method. The env parameter can be used in future to load environment-specific config files like config.qa.properties or config.dev.properties. Currently, it loads the default config.properties file.

```
public static void loadConfig(String env) {
    try (InputStream input = ConfigReader.class.getClassLoader()
        .getResourceAsStream("config/config.properties")) {
        if (input == null) {
            throw new RuntimeException("■ config.properties not found!");
        }
        properties = new Properties();
        properties.load(input);
    } catch (IOException e) {
        throw new RuntimeException("■ Failed to load config.properties file: " + e.getMessage());
    }
}
```

ClassLoader and InputStream

ConfigReader.class.getClassLoader().getResourceAsStream("config/config.properties") loads the file from the classpath (resources folder). This works after Maven copies resources into target/classes or target/test-classes.

```
InputStream input = ConfigReader.class.getClassLoader()
    .getResourceAsStream("config/config.properties");
```

Null check

If the file is missing, input will be null. A RuntimeException is thrown to fail fast with a clear error message.

```
if (input == null) {  
    throw new RuntimeException("■ config.properties file not found in classpath!");  
}
```

Properties loading

This loads key-value pairs from config.properties into the Properties object.

```
properties = new Properties();  
properties.load(input);
```

Error handling

IOException is caught and wrapped into RuntimeException. This is called Exception Wrapping (converting a checked exception into unchecked for simplicity).

```
catch (IOException e) {  
    throw new RuntimeException("■ Failed to load config.properties file: " + e.getMessage(), e);  
}
```

getProperty(String key) method

Fetches a value from the loaded properties. Example usage is shown below.

```
public static String getProperty(String key) {  
    if (properties == null) {  
        throw new RuntimeException("■■ Config file not loaded! Call loadConfig() first.");  
    }  
    return properties.getProperty(key);  
}
```

```
// Usage  
String url = ConfigReader.getProperty("base.url");
```

Interview points

These are the important discussion points for interviews:

- Why use ConfigReader? → Externalizes configuration instead of hardcoding.
- Why use Properties class? → Built-in support for .properties format.
- Why use ClassLoader? → Portable and works in Maven/Gradle builds.
- Why static? → Config values are global and shared across the framework.
- How to handle multiple environments? → Load config-{env}.properties based on input.
- What happens if key is missing? → Returns null, or provide default value.

Summary

ConfigReader is a utility that externalizes test configuration using Java's Properties API and ClassLoader. It ensures framework flexibility, avoids hardcoding, and supports multi-environment

test execution.